AZURE DATA FACTORY

Triggers, Trigger Expressions & Control Flow Activities

Step-by-step practical guide with screenshots and examples

1. What You Will Learn

- What triggers are and how they start Azure Data Factory (ADF) pipeline runs.
- The main trigger types: Schedule, Tumbling Window, Storage Event and Custom Event.
- Common trigger expressions and trigger system variables.
- How to create and attach a trigger to a pipeline.
- How to use control flow activities such as If Condition, ForEach, Until and Switch.
- How Lookup, Get Metadata, Set Variable, Filter, Execute Pipeline and Wait fit into a real pipeline.
- How to validate, debug, publish and monitor a trigger-driven pipeline.

2. Azure Data Factory Basics

Azure Data Factory is a cloud data integration service used to build pipelines that move, transform and orchestrate data. A pipeline contains activities. Triggers determine when a pipeline run should start, while control flow activities determine how the pipeline behaves after it starts.

Concept	Simple meaning	Example
Pipeline	A workflow containing one or more activities.	Copy a CSV file from Blob Storage to another location.
Activity	A single task inside a pipeline.	Copy Data, Lookup, If Condition, ForEach.
Trigger	Starts a pipeline according to a rule or event.	Run every day at 8:00 AM.
Control flow	Controls decisions, loops and dependencies.	If file exists → copy it; otherwise → log/skip.
Expression	Dynamic formula used to calculate a value.	@add(10,10)

3. Creating the Data Factory

The screenshot below shows the final Review + create page used while creating an Azure Data Factory resource.

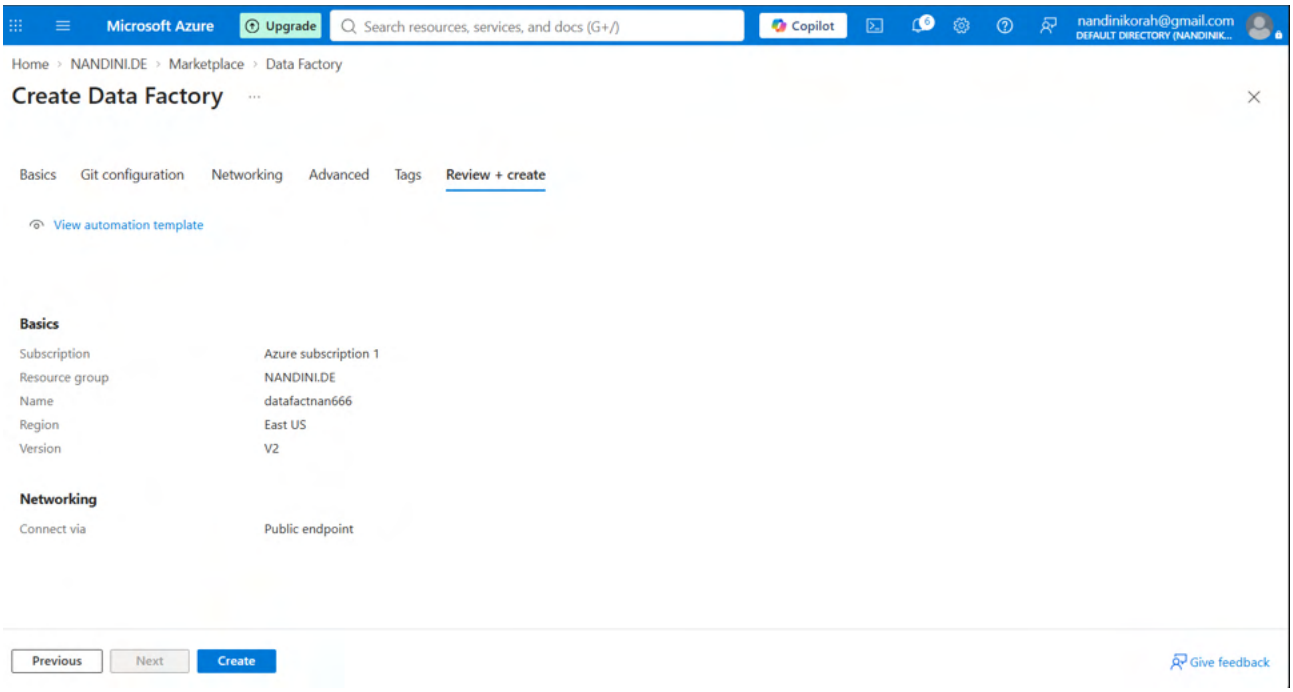
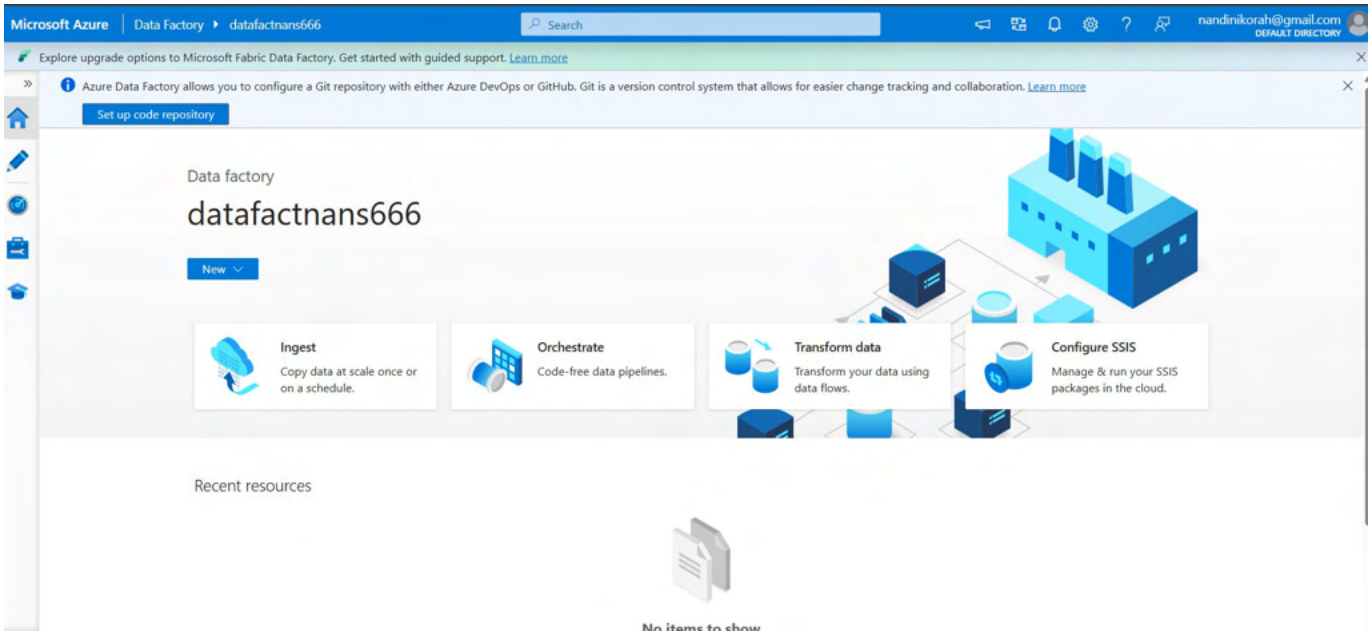


Figure 1. Azure Data Factory — Review + create. The displayed example uses East US, V2 and a public endpoint.

1. Open the Azure portal and search for Data Factory.
2. Select Create Data Factory.
3. Choose the Azure subscription and resource group.
4. Enter a globally unique Data Factory name.
5. Choose a region and review networking/Git settings as required.
6. Open Review + create and verify the settings.
7. Select Create and wait for deployment to finish.
8. Open the Data Factory Studio and move to the Author/Integrate area to build pipelines.



4. Triggers in Azure Data Factory

A trigger is the mechanism that causes a pipeline to run. A trigger is not the same thing as an activity: the trigger starts the pipeline, and the activities perform the work.

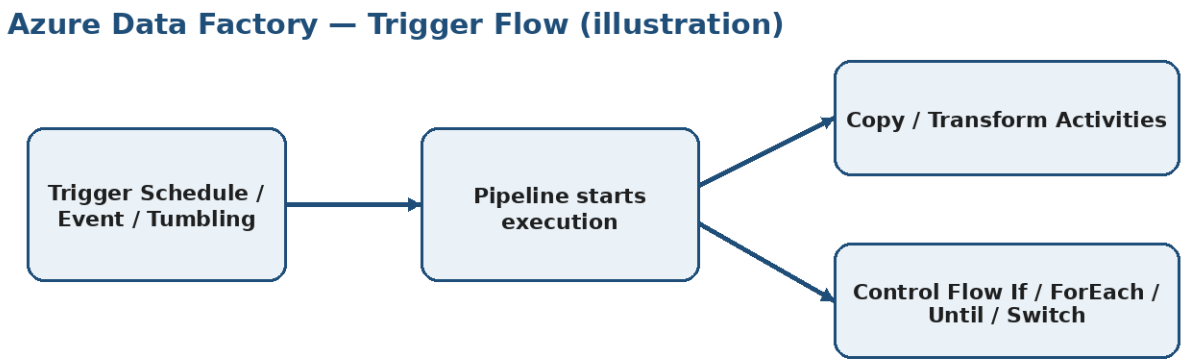


Figure 2. Trigger-to-pipeline flow — illustrative diagram.

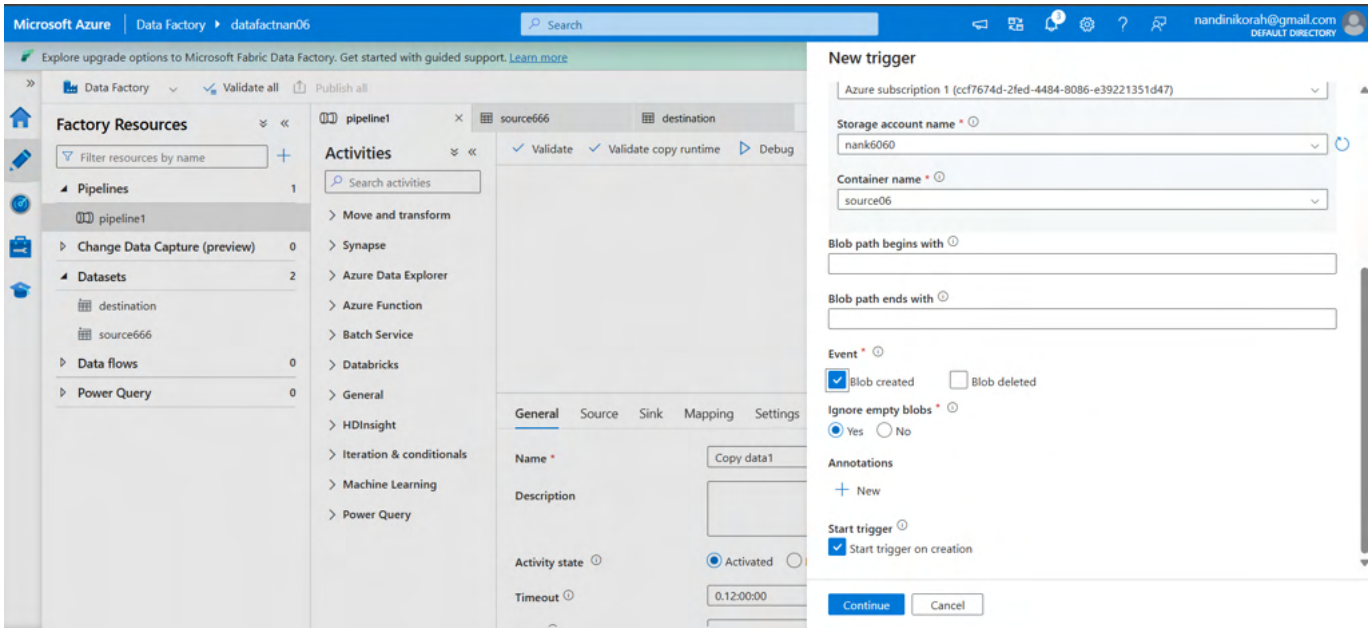
Trigger type	When to use it	Typical example
Schedule trigger	Run at a recurring time or interval.	Run a daily sales load at 06:00.
Tumbling window trigger	Process fixed, contiguous time windows and retain window state.	Process each hourly data slice; support dependency/retry/backfill scenarios.
Storage event trigger	Start when a supported storage event occurs.	A new file arrives in Azure Storage/Blob Storage.
Custom event trigger	Start when a custom event is published through the supported event mechanism.	Start processing when an upstream system publishes an event.

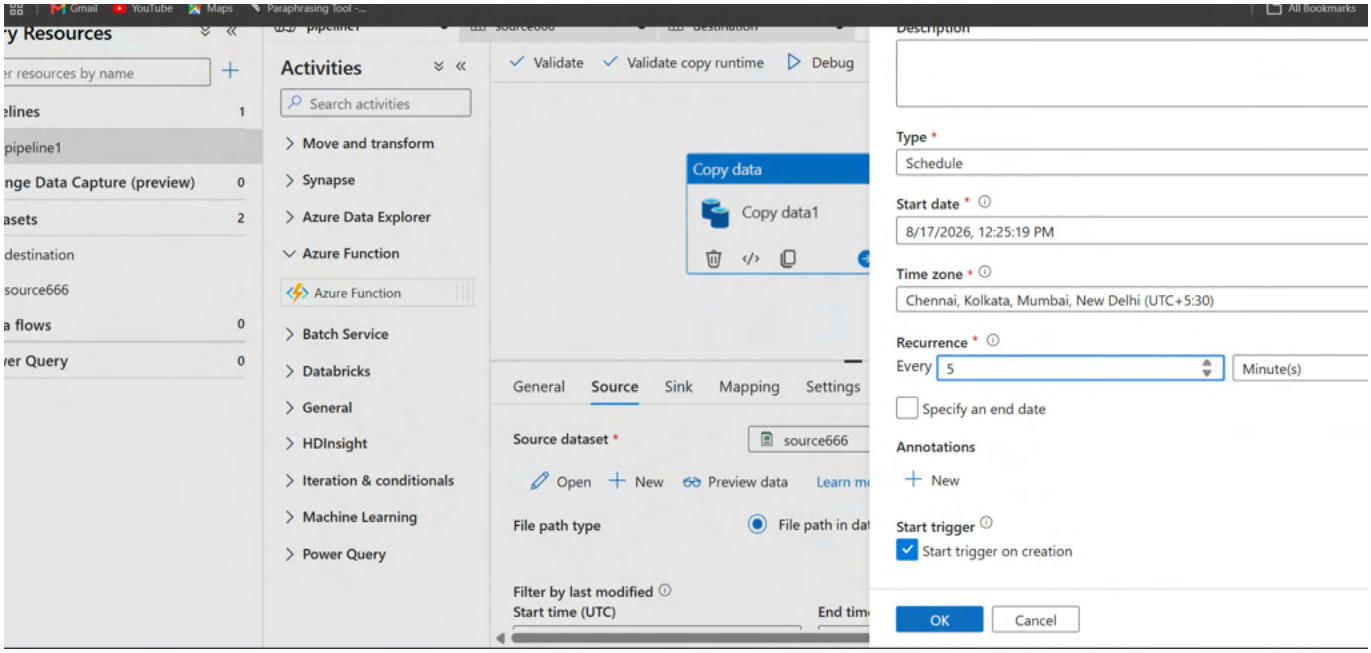
5. Schedule Trigger — Step by Step

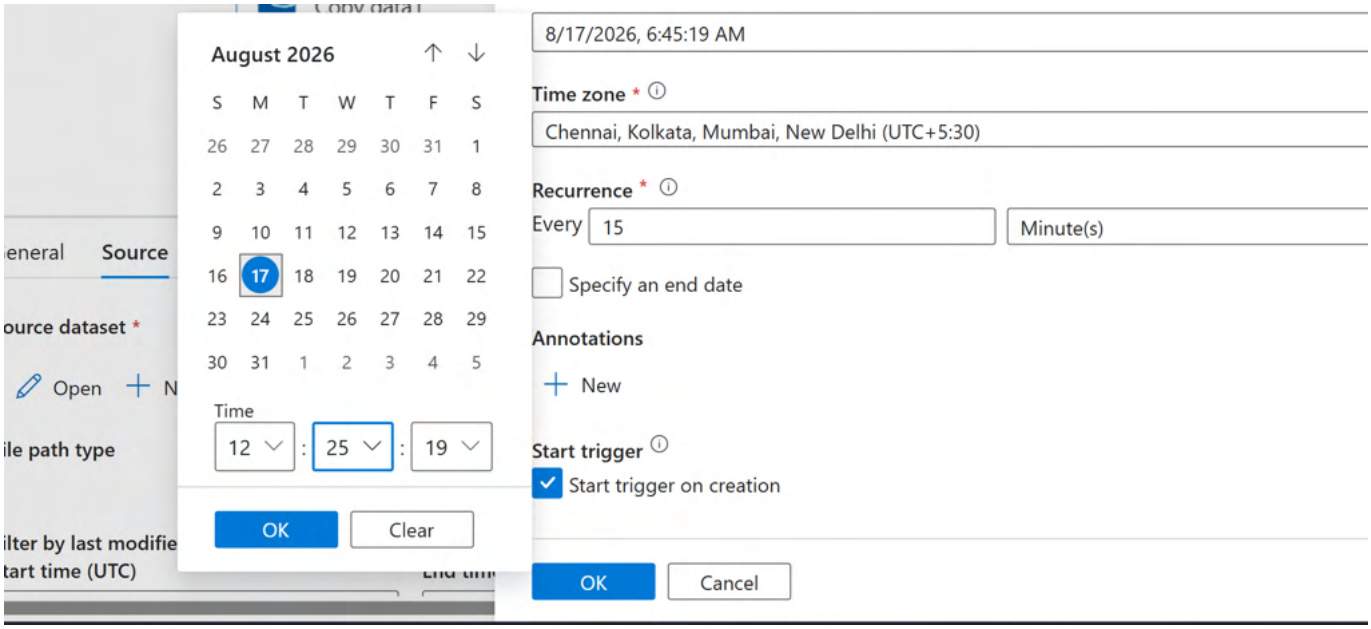
A Schedule trigger is the easiest trigger to learn. It runs a pipeline on a recurring schedule such as every 15 minutes, hourly, daily, weekly or monthly.

9. Open your pipeline in the Data Factory Studio.
10. On the pipeline toolbar, select Add trigger → New/Edit.
11. Select + New to create a trigger.
12. Give the trigger a meaningful name, for example DailySalesTrigger.
13. Choose Schedule as the trigger type.
14. Set the start date/time and time zone.
15. Set the recurrence, for example Every 1 Day or Every 15 Minutes.
16. Optionally specify an end date.
17. Confirm the pipeline and trigger parameters.
18. Select OK/Finish and then Publish all.
19. Start the trigger if it is not configured to start automatically.
20. Check Monitor → Trigger runs and Pipeline runs.

Important: saving a trigger in the authoring UI does not by itself make the published trigger run. Publish the changes and ensure the trigger is started/activated.


Microsoft Azure
Data Factory
datafactnan06
Search
nandinikorah@gmail.com
DEFAULT DIRECTORY
Explore upgrade options to Microsoft Fabric Data Factory. Get started with guided support. Learn more
Data Factory
Validate all
Publish all
Factory Resources
Filter resources by name
Pipelines 1
pipeline1
Change Data Capture (preview) 0
Datasets 2
destination
source666
Data flows 0
Power Query 0
pipeline1
source666
destination
Activities
Search activities
Move and transform
Synapse
Azure Data Explorer
Azure Function
Batch Service
Databricks
General
HDInsight
Iteration & conditionals
Machine Learning
Power Query
Validate
Validate copy runtime
Debug
General
Source
Sink
Mapping
Settings
Name *
Copy data1
Description
Activity state
Activated
Timeout
0.12:00:00
New trigger
Azure subscription 1 (ccf7674d-2fed-4484-8086-e39221351d47)
Storage account name *
nank6060
Container name *
source06
Blob path begins with
Blob path ends with
Event *
Blob created
Blob deleted
Ignore empty blobs *
Yes
No
Annotations
New
Start trigger
Start trigger on creation
Continue
Cancel

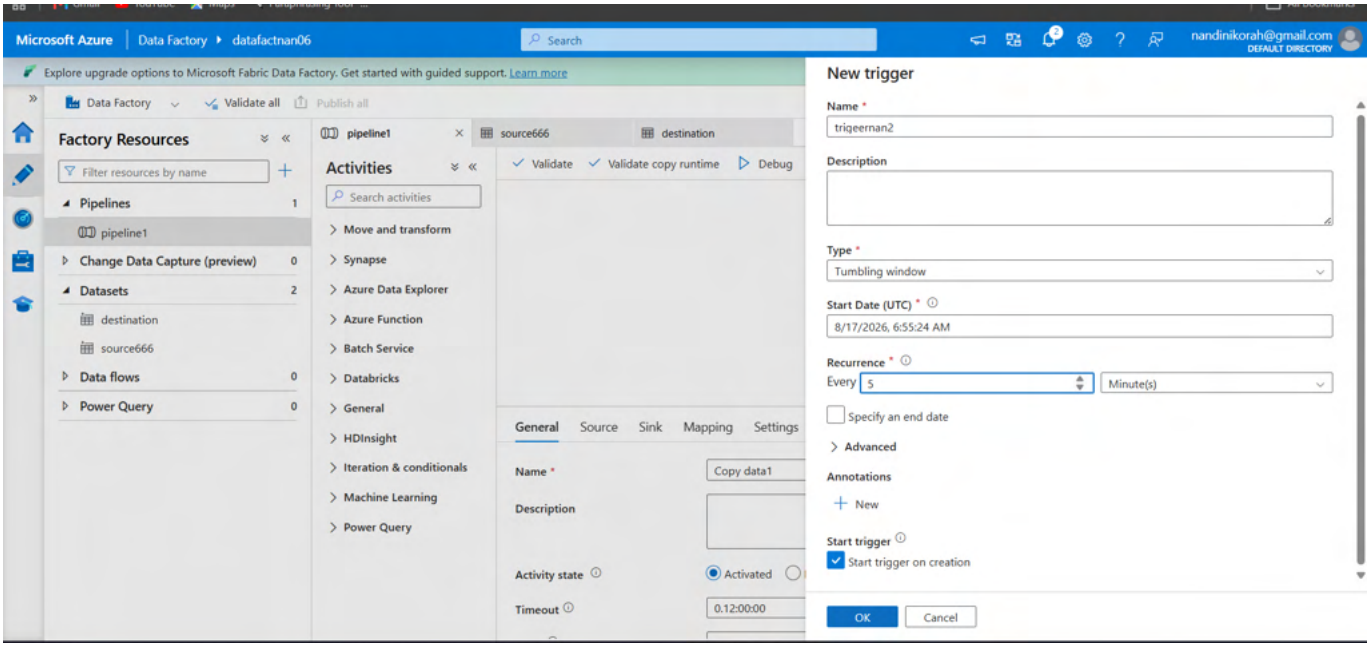

Activities
Search activities
Move and transform
Synapse
Azure Data Explorer
Azure Function
Azure Function
Batch Service
Databricks
General
HDInsight
Iteration & conditionals
Machine Learning
Power Query
Validate
Validate copy runtime
Debug
Copy data
Copy data1
General
Source
Sink
Mapping
Settings
Source dataset *
source666
Open
New
Preview data
File path type
Filter by last modified
Start time (UTC)
Type *
Schedule
Start date *
8/17/2026, 12:25:19 PM
Time zone *
Chennai, Kolkata, Mumbai, New Delhi (UTC+5:30)
Recurrence *
Every
5
Minute(s)
Specify an end date
Annotations
New
Start trigger
Start trigger on creation
OK
Cancel

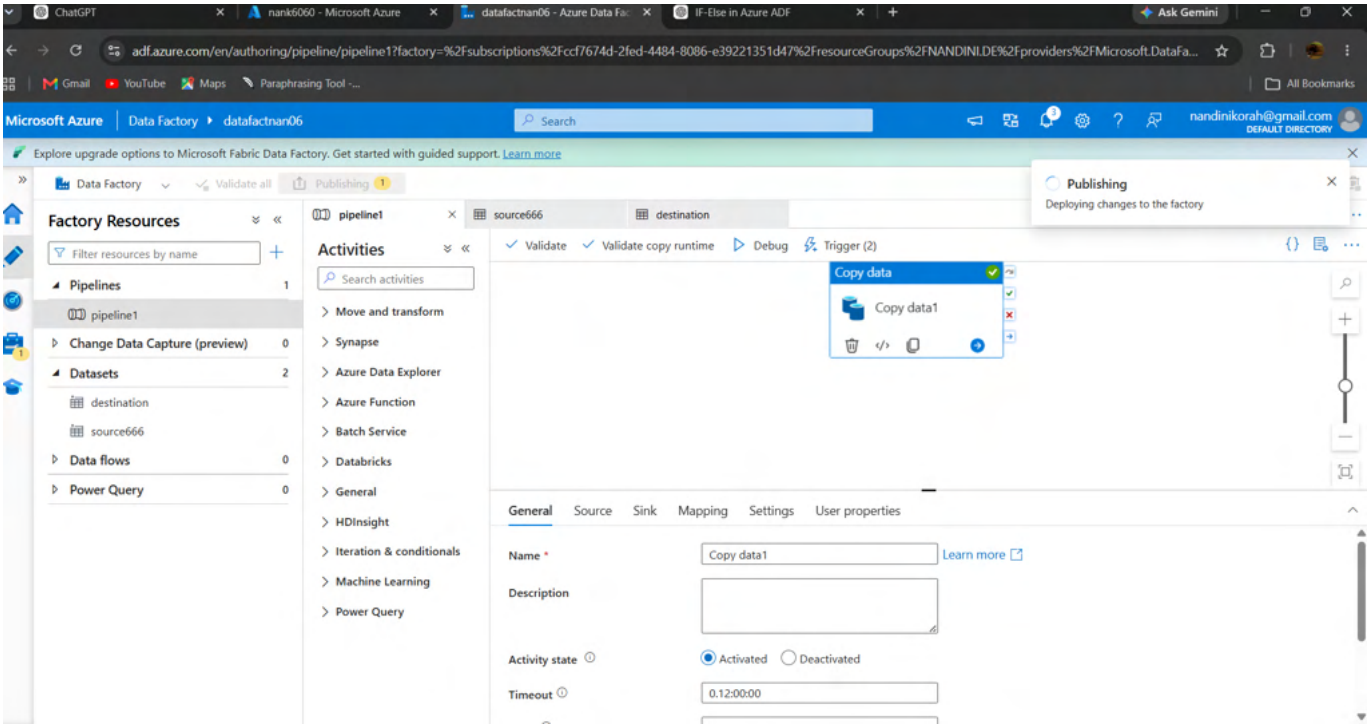


6. Tumbling Window Trigger

A Tumbling Window trigger fires at fixed, non-overlapping, contiguous intervals and retains state. It is useful when each time window represents a data slice that must be processed reliably.

- Choose Tumbling window when creating a new trigger.
- Define the frequency and interval, such as every 1 hour.
- Set the start time and optional end time.
- Configure delay if the pipeline should wait after the window is due.
- Use max concurrency when several ready windows may be processed in parallel.
- Use trigger dependencies when one tumbling window trigger must wait for another.
- Use retry settings when automatic retries are required.

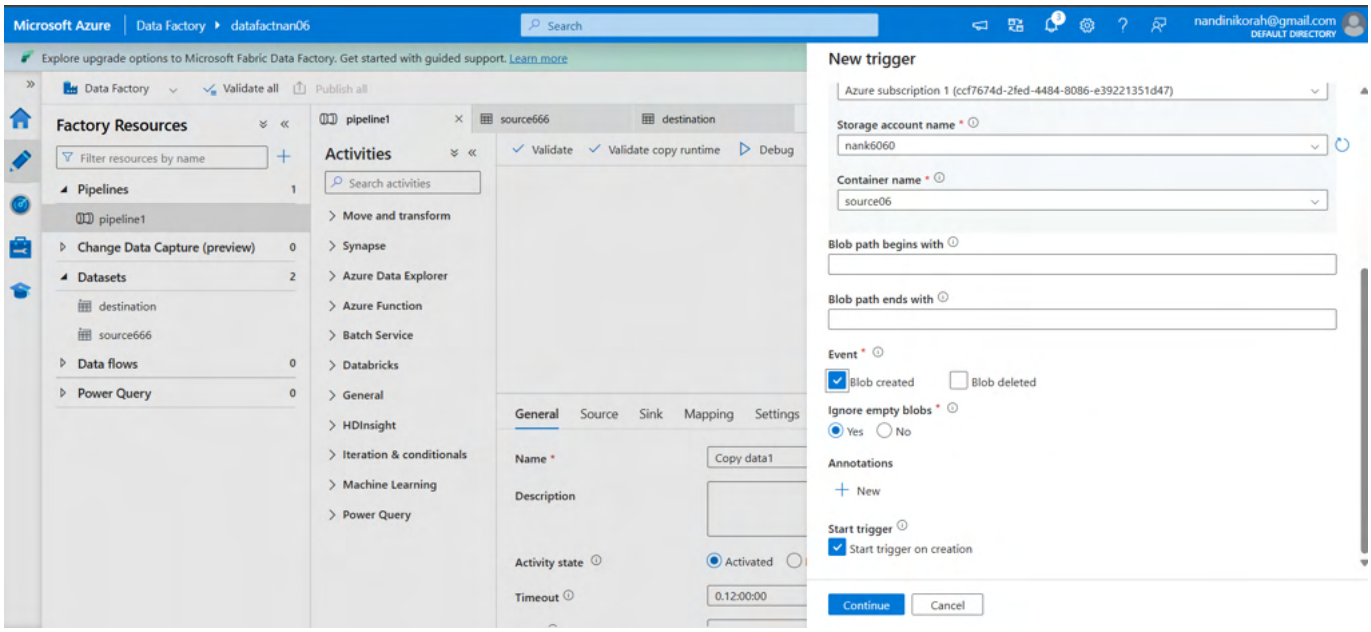




7. Event-Based Triggers

Event-based triggers are useful when time is not the correct signal. Instead of asking the pipeline to check every hour, the pipeline can start when a relevant event occurs.

- Storage event trigger: useful for file arrival or supported storage events.
- Custom event trigger: useful when an external system publishes a custom event containing information needed by the pipeline.
- Event-driven designs can reduce unnecessary polling and can make ingestion more responsive.



8. Trigger Expressions and System Variables

ADF expressions are used to calculate dynamic values. The Expression Builder provides functions, parameters, system variables and pipeline variables.

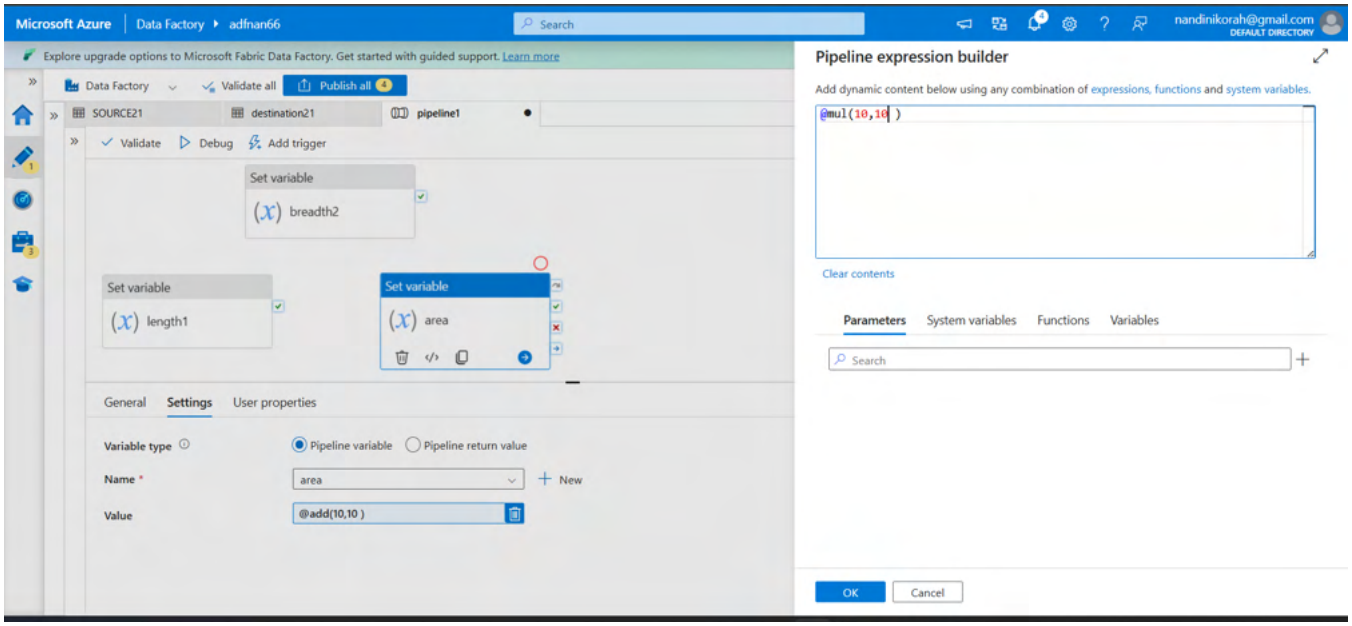


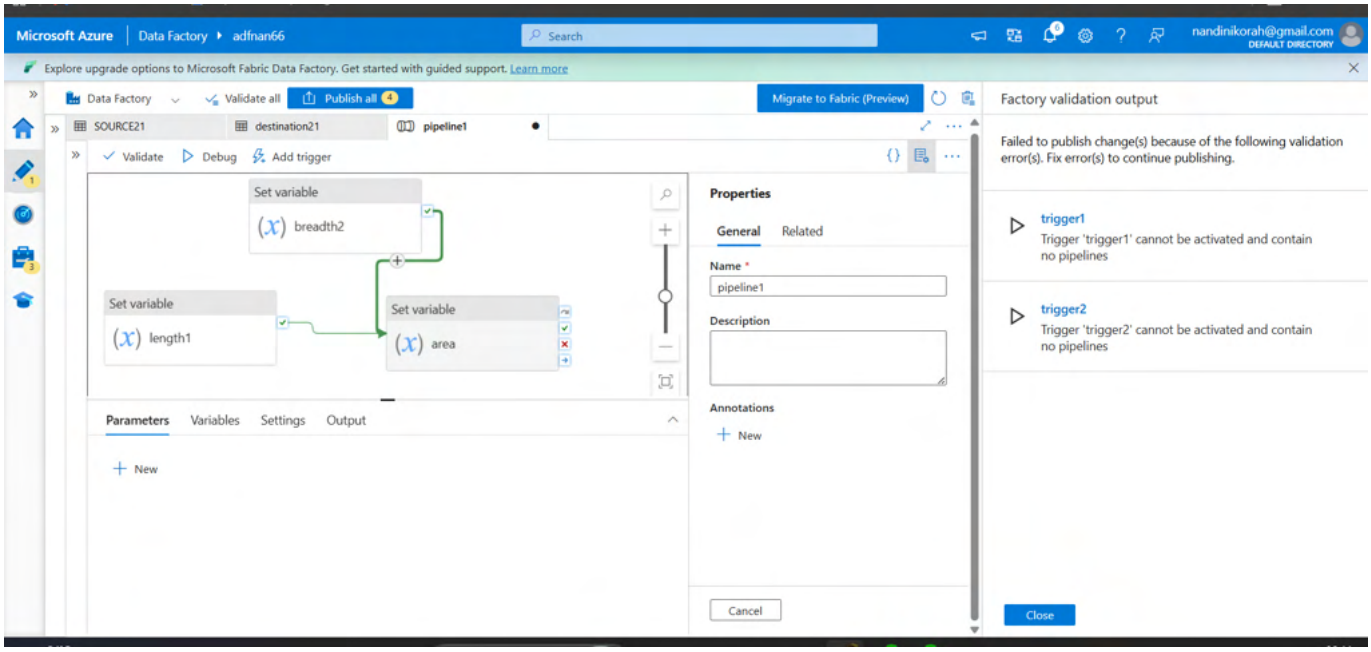
Figure 3. User-provided screenshot: Pipeline Expression Builder with a Set Variable activity. The example shows @add(10,10) in the activity and @mul(10,10) in the expression editor.

Expression / variable	Purpose	Example use
@trigger().scheduledTime	Scheduled time of a schedule trigger.	Pass scheduled time to a pipeline parameter.
@trigger().startTime	Time the trigger actually fired.	Audit/log the trigger execution time.
@trigger().outputs.windowStartTime	Start of a tumbling-window interval.	Build a source folder path for that time slice.
@trigger().outputs.windowEndTime	End of a tumbling-window interval.	Build an end boundary for incremental processing.
@triggerBody().fileName	File name from a supported storage event trigger.	Use the arriving file name in a dataset/path.
@add(10,10)	Adds numeric values.	Set a variable to 20.
@mul(10,10)	Multiplies numeric values.	Set a variable to 100.
@variables('area')	Reads a pipeline variable.	Use a previously calculated value.

9. How to Use the Expression Builder

21. Select the activity that needs a dynamic value, such as Set Variable.
22. Click inside the value field.
23. Choose Add dynamic content.
24. The Pipeline Expression Builder opens.
25. Use the Functions tab for built-in functions, Variables for pipeline variables, and System variables where applicable.
26. Enter or select the expression.
27. Select OK and confirm the expression appears in the activity field.
28. Validate the pipeline before debugging.

Example: @add(10,10) returns 20. Example: @mul(10,10) returns 100. In the supplied screenshot, the Set Variable activity named area is configured with @add(10,10).



10. Useful Expression Functions

Function	Example	What it does
add	@add(10,5)	Adds numbers.
sub	@sub(10,5)	Subtracts one number from another.
mul	@mul(10,5)	Multiplies numbers.
div	@div(10,5)	Divides numbers.
equals	@equals(variables('status'),'Ready')	Checks equality.
and	@and(equals(...),equals(...))	Requires both conditions to be true.
or	@or(equals(...),equals(...))	Requires at least one condition to be true.
contains	@contains('AzureDataFactory','Data')	Checks whether text/array contains a value.
concat	@concat('sales','_','2026')	Combines text values.
formatDateTime	@formatDateTime(utcnow(),'yyyy-MM-dd')	Formats a date/time.
if	@if(equals(...),'Yes','No')	Returns one value or another based on a condition.

11. Control Flow Activities

Control flow activities decide what happens inside a pipeline after it starts. They are especially important for branching, looping, validation, variables and orchestration.

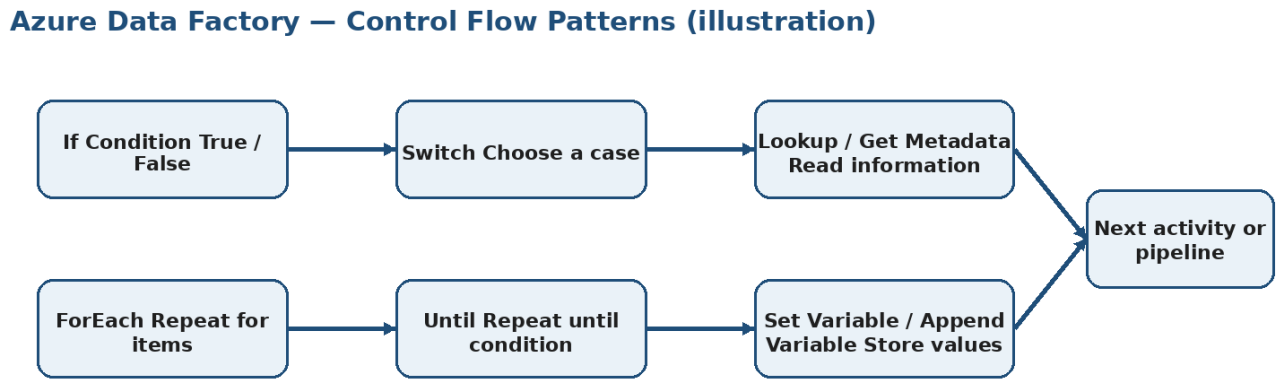


Figure 4. Control flow patterns — illustrative diagram.

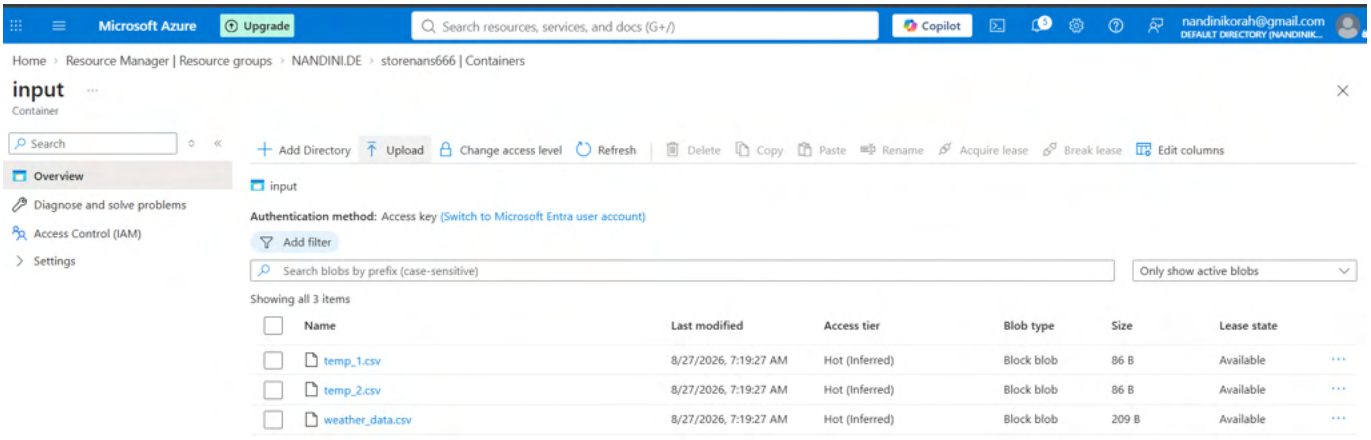
Activity	Purpose	Simple example
If Condition	Branch into True and False paths.	If file exists → copy; else → log/skip.
ForEach	Repeat activities for every item in an array.	Process every file name returned by Lookup.
Until	Repeat until a condition becomes true.	Keep checking until a file/status is ready.
Switch	Choose one case from multiple possible values.	Country = IN → India path; US → US path.
Filter	Filter an input array using an expression.	Keep only files ending in .csv.
Lookup	Read a record/table/value and expose its output.	Read a list of files or configuration values.
Get Metadata	Retrieve metadata such as exists, size, structure or child items.	Check whether a file exists.
Set Variable	Assign a value to an existing variable.	Set status = 'Ready'.
Append Variable	Add a value to an array variable.	Append each processed file name.
Execute Pipeline	Invoke another pipeline.	Parent pipeline calls a reusable child pipeline.
Wait	Pause the pipeline for a specified period.	Wait 30 seconds before polling again.
Web / Webhook	Call an endpoint or wait for a callback.	Send a notification or integrate with an API.
Validation	Ensure a referenced dataset/resource meets a condition before continuing.	Validate that an expected dataset exists.

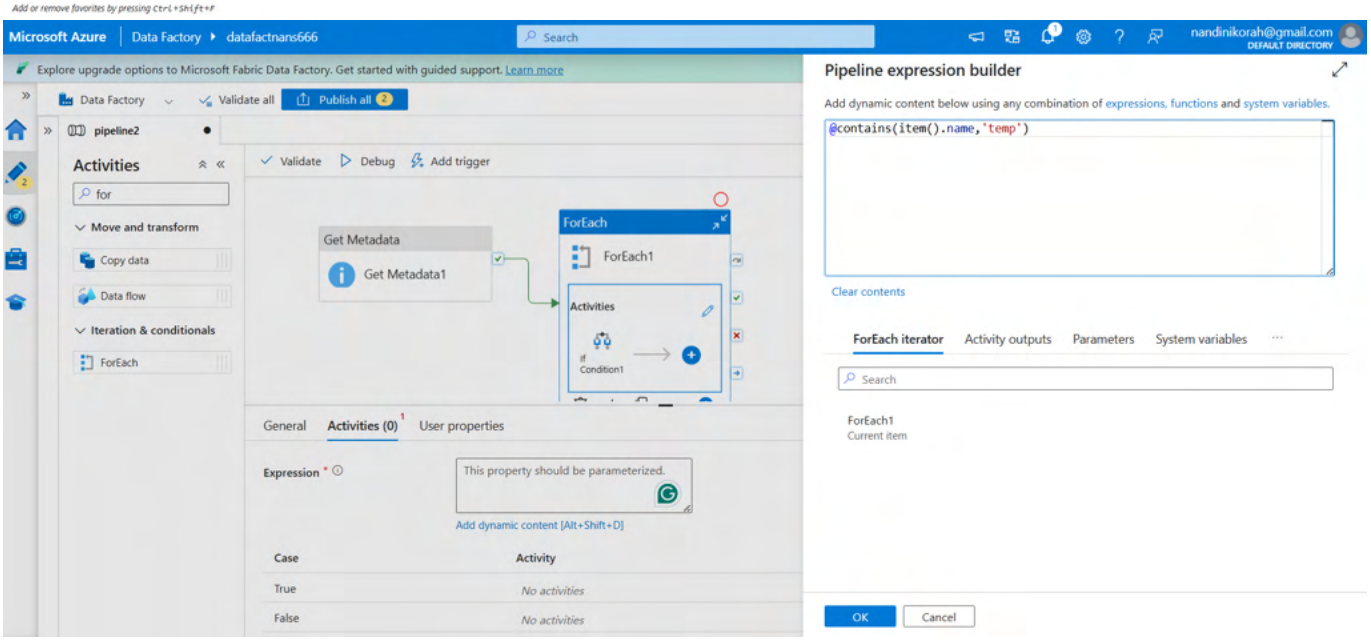
12. If Condition — Step by Step

If Condition is the ADF equivalent of an if/else decision.

29. Open the pipeline and locate the Activities pane.
30. Under Iteration & conditionals, drag If Condition onto the canvas.
31. Select the activity and open Settings.
32. In Expression, select Add dynamic content.
33. Create a condition.
34. Add activities inside the True branch for the condition being true.
35. Add activities inside the False branch for the condition being false.
36. Connect the previous activity to If Condition with the appropriate dependency path.

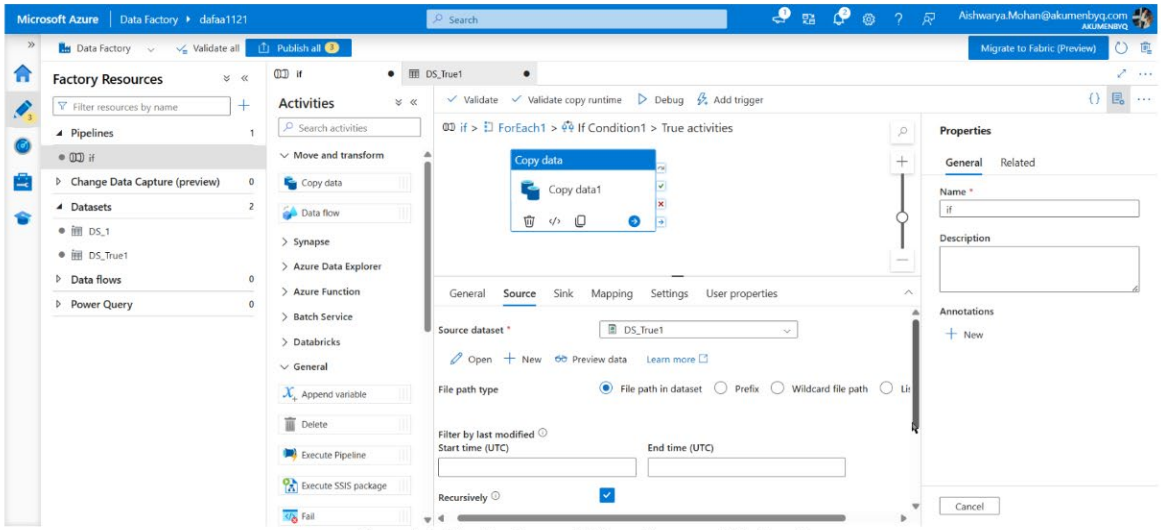
37. Validate and debug the pipeline.





⑩ Replace Set Variable with Copy Data

True → Copy data1 | False → Copy data2



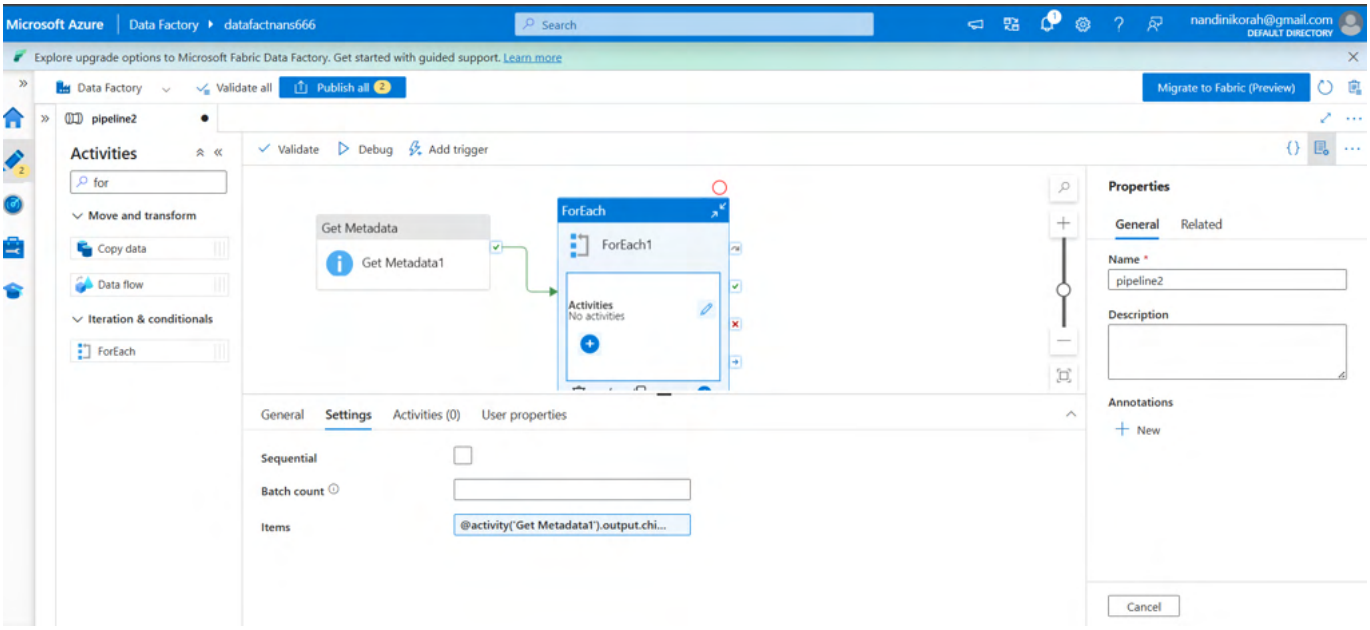
Copy data1 inside True activities – Source = DS_True1

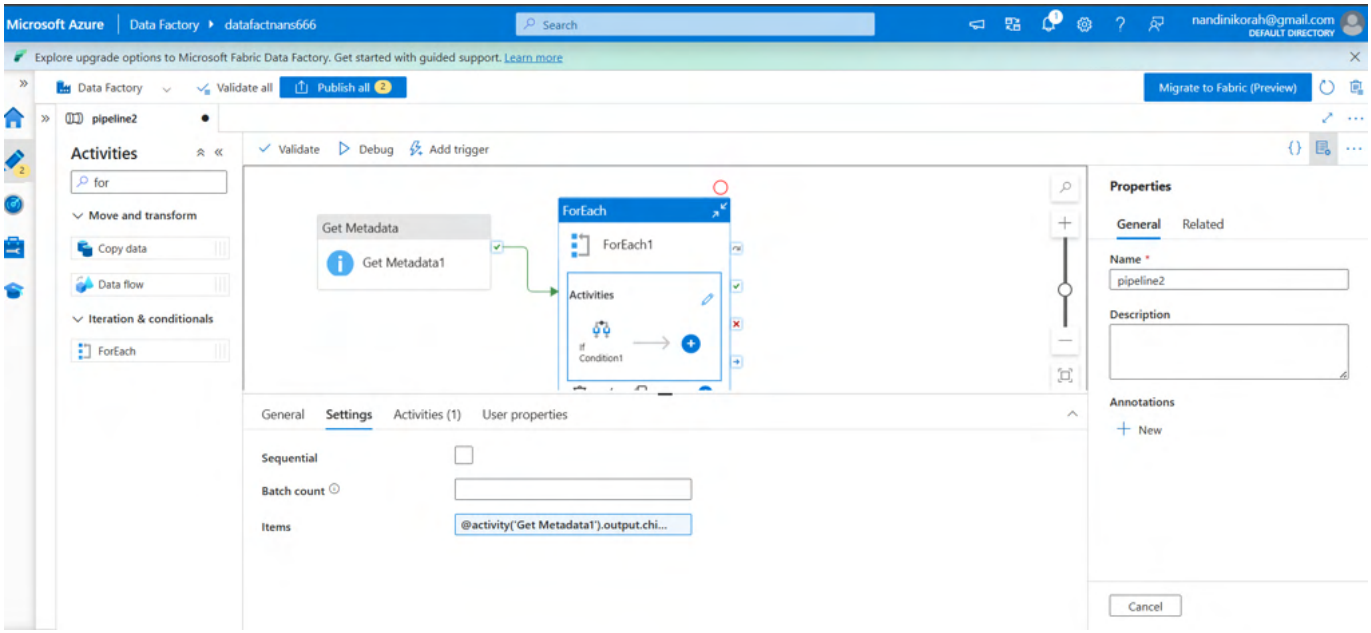
13. ForEach — Step by Step

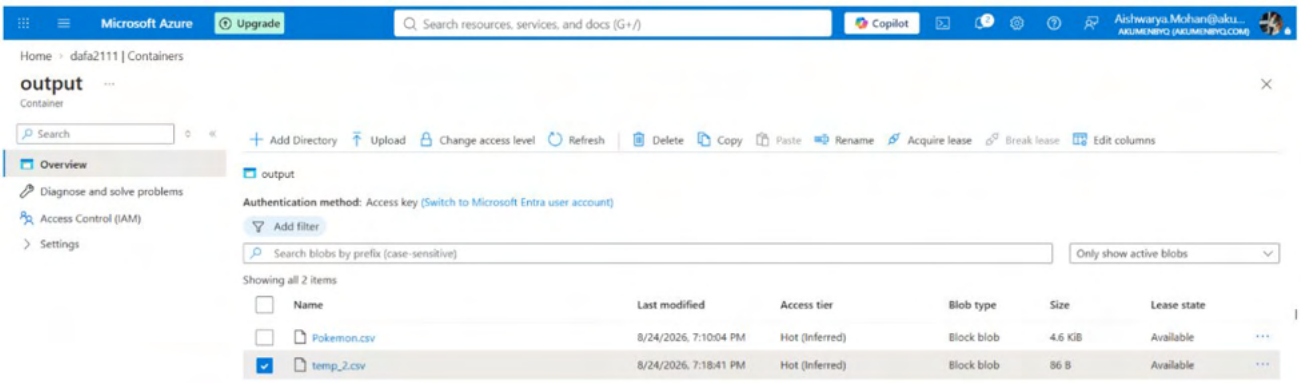
ForEach repeats one or more activities for every item in an input array.

38. Add a Lookup, Get Metadata or other activity that produces an array.
39. Drag ForEach onto the pipeline canvas.
40. In Settings → Items, select Add dynamic content.
41. Select the array output, for example @activity('Lookup1').output.value.
42. Open the ForEach Activities editor.
43. Add the activity or activities that should run for each item.
44. Inside the loop, use @item() to refer to the current item.
45. Choose sequential execution when order matters; otherwise use controlled parallelism when appropriate.

```
@activity('Lookup1').output.value
@item()
```


Microsoft Azure
Data Factory
datafactnans666
Search
nandinikorah@gmail.com
DEFAULT DIRECTORY
Explore upgrade options to Microsoft Fabric Data Factory. Get started with guided support. Learn more
Data Factory
Validate all
Publish all 2
Migrate to Fabric (Preview)
pipeline2
Activities
for
Move and transform
Copy data
Data flow
Iteration & conditionals
ForEach
Validate
Debug
Add trigger
Get Metadata
Get Metadata1
ForEach
ForEach1
Activities
No activities
Properties
General
Related
Name *
pipeline2
Description
Annotations
New
General
Settings
Activities (0)
User properties
Sequential
Batch count
Items
@activity('Get Metadata1').output.chi...
Cancel


Microsoft Azure
Data Factory
datafactnans666
Search
nandinikorah@gmail.com
DEFAULT DIRECTORY
Explore upgrade options to Microsoft Fabric Data Factory. Get started with guided support. Learn more
Data Factory
Validate all
Publish all 2
Migrate to Fabric (Preview)
pipeline2
Activities
for
Move and transform
Copy data
Data flow
Iteration & conditionals
ForEach
Validate
Debug
Add trigger
Get Metadata
Get Metadata1
ForEach
ForEach1
Activities
If Condition1
Properties
General
Related
Name *
pipeline2
Description
Annotations
New
General
Settings
Activities (1)
User properties
Sequential
Batch count
Items
@activity('Get Metadata1').output.chi...
Cancel



14. Supporting Control Flow Activities

14.1 Lookup

Lookup reads a value, record or set of records from an external source. Its output can feed later activities such as ForEach, Copy or conditional logic.

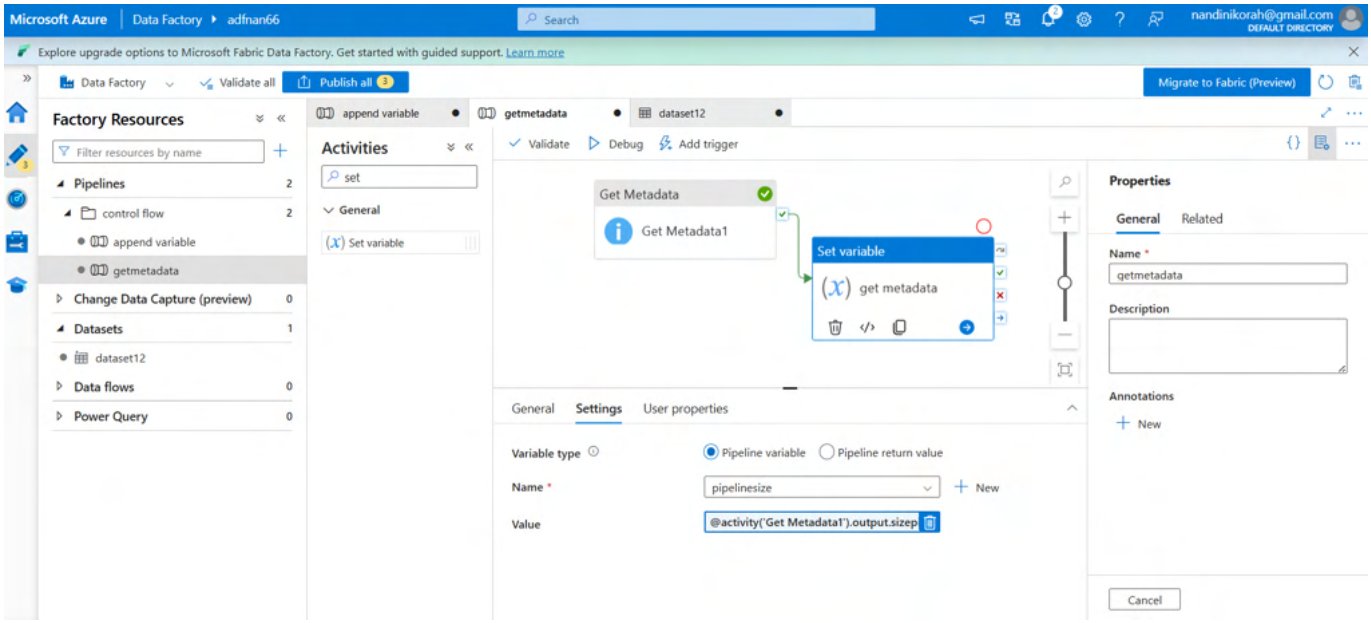
- Example: read a configuration table containing file names.
- Use the Lookup output as the Items input of ForEach.
- For multiple rows, use the array output such as @activity('Lookup1').output.value.

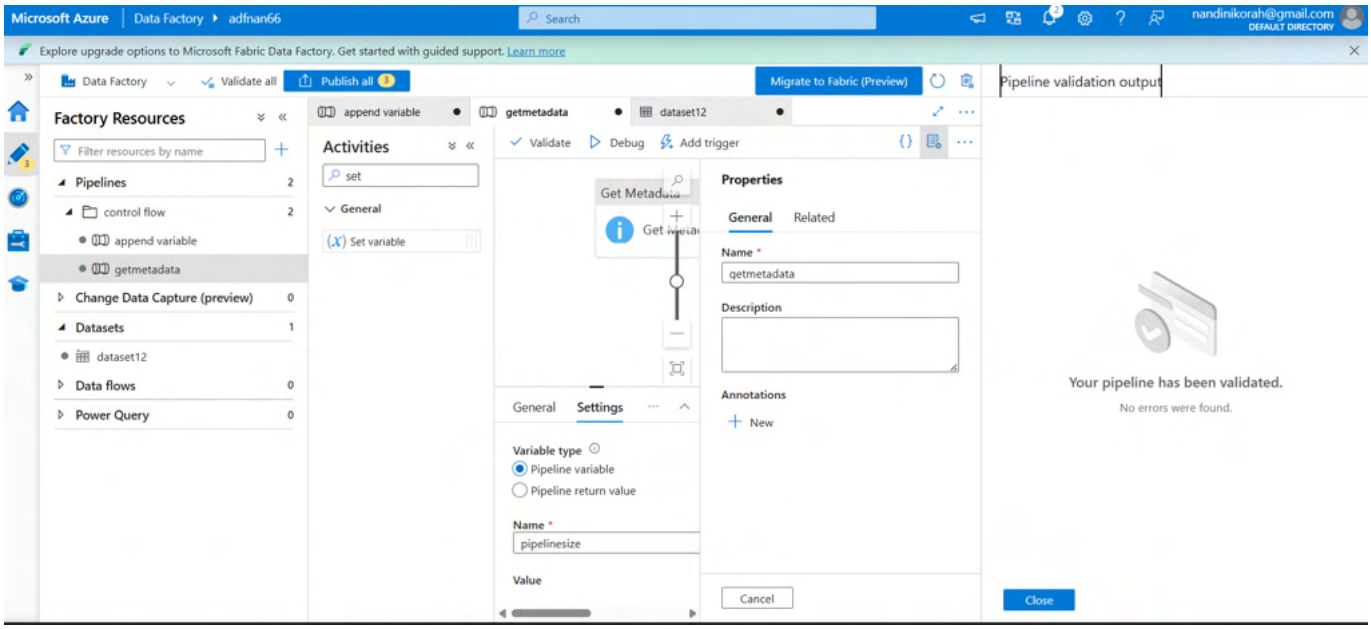
14.2 Get Metadata

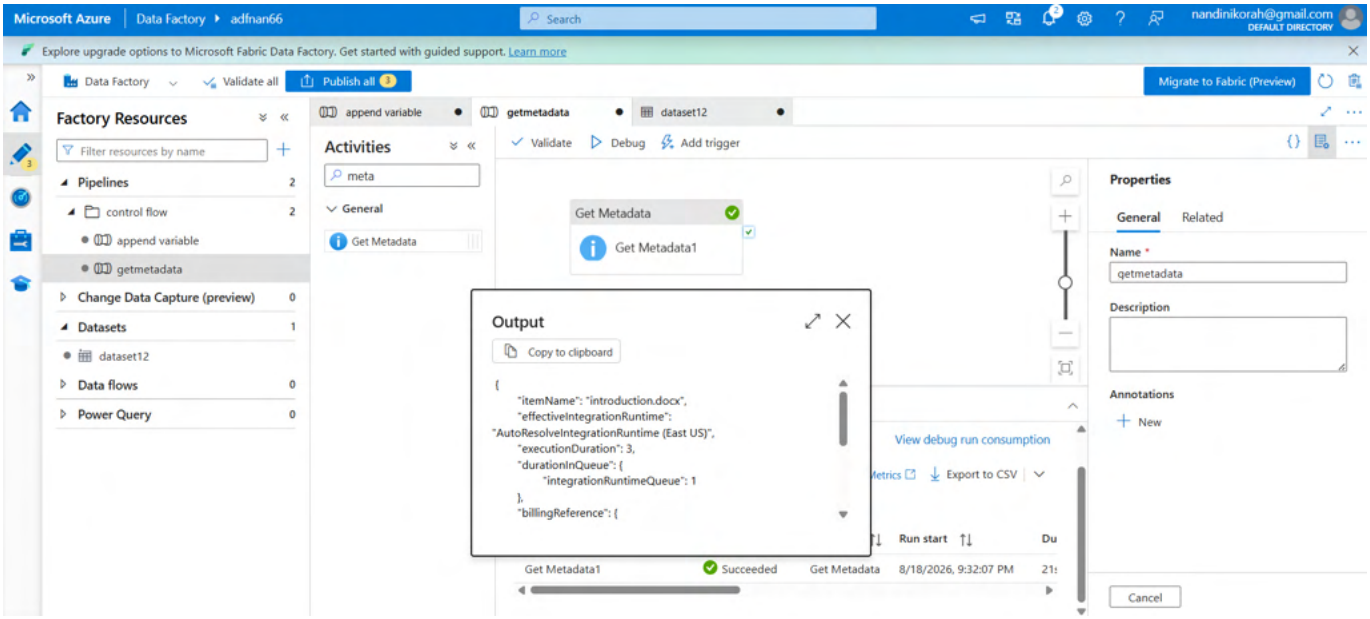
Get Metadata retrieves information about a dataset/file/folder. It is often paired with If Condition.

- Common checks include file existence, size and child items.
- Example pattern: Get Metadata → If Condition → Copy Data.

```
@activity('Get Metadata1').output.exists
```


Microsoft Azure
Data Factory
adfnan66
Search
nandinikorah@gmail.com
DEFAULT DIRECTORY
Explore upgrade options to Microsoft Fabric Data Factory. Get started with guided support. Learn more
Data Factory
Validate all
Publish all
Migrate to Fabric (Preview)
Factory Resources
Filter resources by name
Pipelines
control flow
append variable
getmetadata
Change Data Capture (preview)
Datasets
dataset12
Data flows
Power Query
Activities
set
General
Set variable
Validate
Debug
Add trigger
Get Metadata
Get Metadata1
Set variable
get metadata
Properties
General
Related
Name *
getmetadata
Description
Annotations
New
General
Settings
User properties
Variable type
Pipeline variable
Pipeline return value
Name *
pipelinesize
New
Value
@activity('Get Metadata1').output.sizep
Cancel


Microsoft Azure
Data Factory
adfnan66
Search
nandinikorah@gmail.com
DEFAULT DIRECTORY
Explore upgrade options to Microsoft Fabric Data Factory. Get started with guided support. Learn more
Data Factory
Validate all
Publish all
Migrate to Fabric (Preview)
Pipeline validation output
Factory Resources
Filter resources by name
Pipelines
control flow
append variable
getmetadata
Change Data Capture (preview)
Datasets
dataset12
Data flows
Power Query
Activities
set
General
Set variable
Validate
Debug
Add trigger
Properties
General
Related
Name *
getmetadata
Description
Annotations
New
General
Settings
Variable type
Pipeline variable
Pipeline return value
Name *
pipelinesize
Value
Cancel
Close
Your pipeline has been validated.
No errors were found.

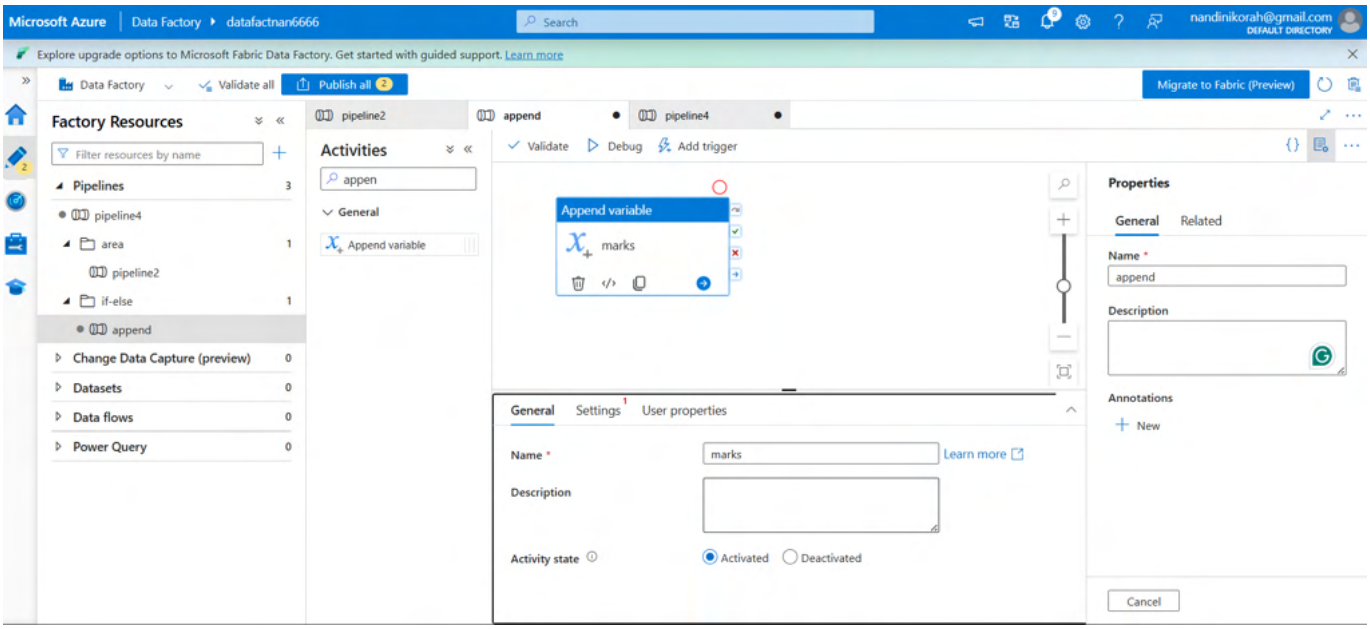


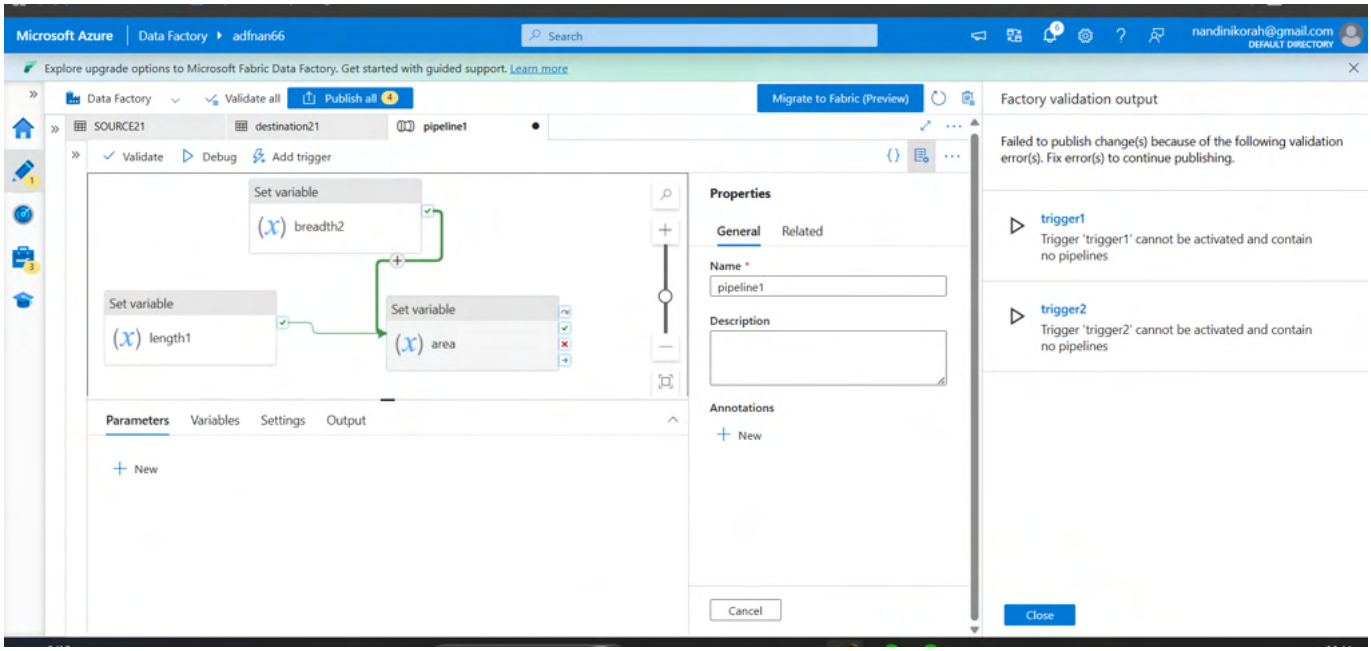
14.3 Set Variable and Append Variable

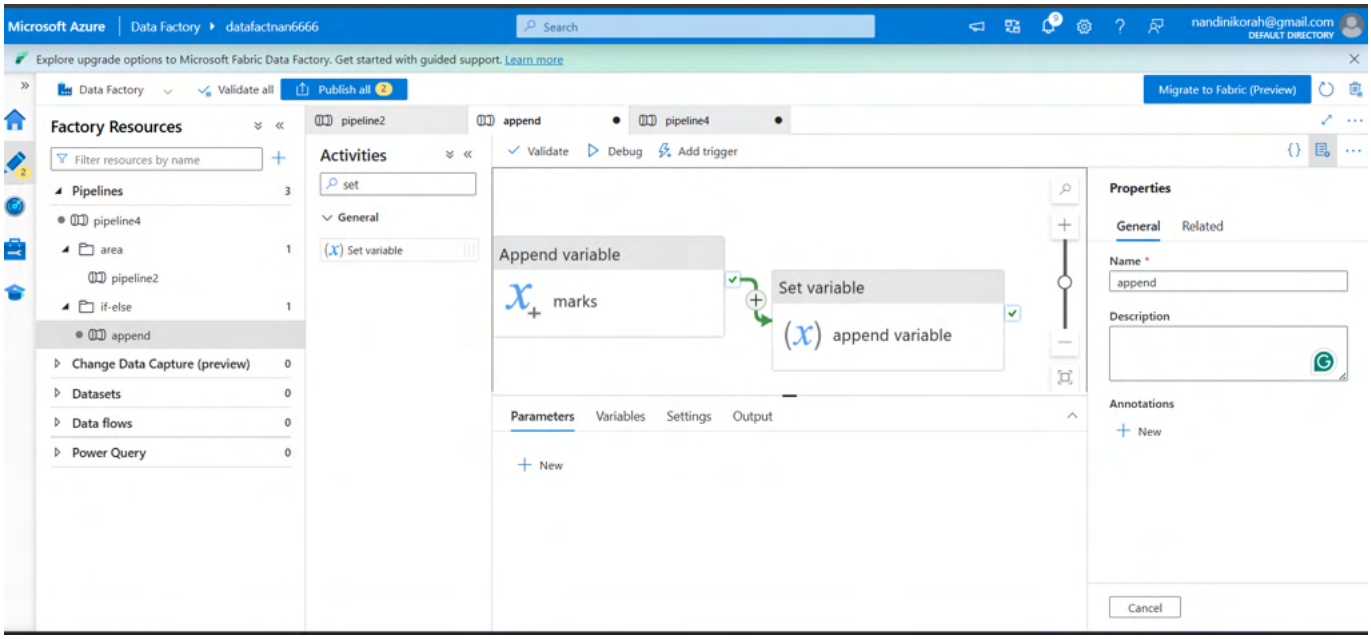
Set Variable assigns a value to an existing variable. Append Variable adds an item to an array variable.

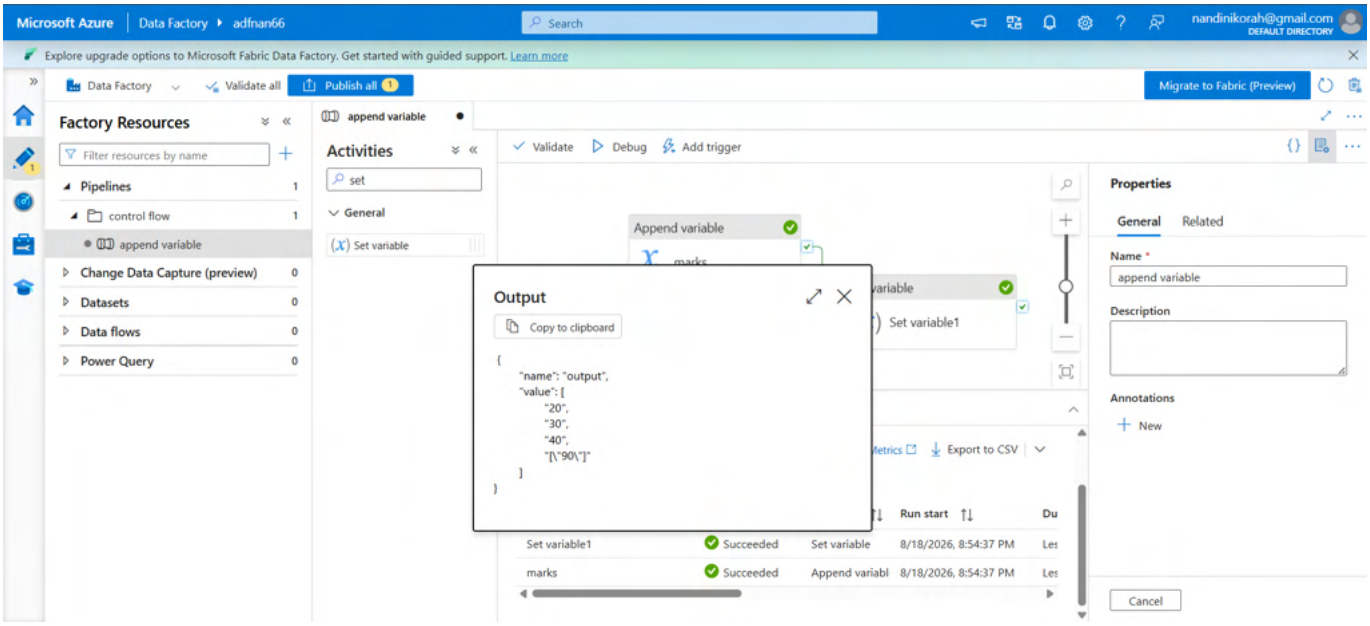
- Use variables to store status, counters, paths or intermediate values.
- Use array variables with Append Variable when you need to collect multiple results.

```
@add(10,10)
@mul(10,10)
```




Microsoft Azure
Data Factory ▸ adfnan66
Search
nandinikorah@gmail.com
DEFAULT DIRECTORY
Explore upgrade options to Microsoft Fabric Data Factory. Get started with guided support. Learn more
Data Factory
Validate all
Publish all 4
Migrate to Fabric (Preview)
SOURCE21
destination21
pipeline1
Validate
Debug
Add trigger
Set variable
breadth2
Set variable
length1
Set variable
area
Parameters
Variables
Settings
Output
New
Properties
General
Related
Name *
pipeline1
Description
Annotations
New
Cancel
Factory validation output
Failed to publish change(s) because of the following validation error(s). Fix error(s) to continue publishing.
trigger1
Trigger 'trigger1' cannot be activated and contain no pipelines
trigger2
Trigger 'trigger2' cannot be activated and contain no pipelines
Close


Microsoft Azure
Data Factory ▸ datafactnan6666
Search
nandinikorah@gmail.com
DEFAULT DIRECTORY
Explore upgrade options to Microsoft Fabric Data Factory. Get started with guided support. Learn more
Data Factory
Validate all
Publish all 2
Migrate to Fabric (Preview)
Factory Resources
Filter resources by name
Pipelines 3
pipeline4
area 1
pipeline2
if-else 1
append
Change Data Capture (preview) 0
Datasets 0
Data flows 0
Power Query 0
pipeline2
append
pipeline4
Activities
set
General
Set variable
Validate
Debug
Add trigger
Append variable
marks
Set variable
append variable
Parameters
Variables
Settings
Output
New
Properties
General
Related
Name *
append
Description
Annotations
New
Cancel



15. End-to-End Example: File Arrival Processing

A practical beginner-to-intermediate pipeline can combine a trigger, Get Metadata, If Condition, ForEach, Set Variable and Copy Data.

Step	ADF component	Logic
1	Storage/Event trigger	Start when a supported file event occurs.
2	Get Metadata	Check file/folder information.
3	If Condition	Continue only when the expected file exists.
4	Set Variable	Store the file path or processing status.
5	Copy Data	Move/copy the file to the destination.
6	ForEach (optional)	Process each file when the input is a list/array.
7	Execute Pipeline (optional)	Call a reusable transformation pipeline.
8	Monitor	Check pipeline and trigger run status.

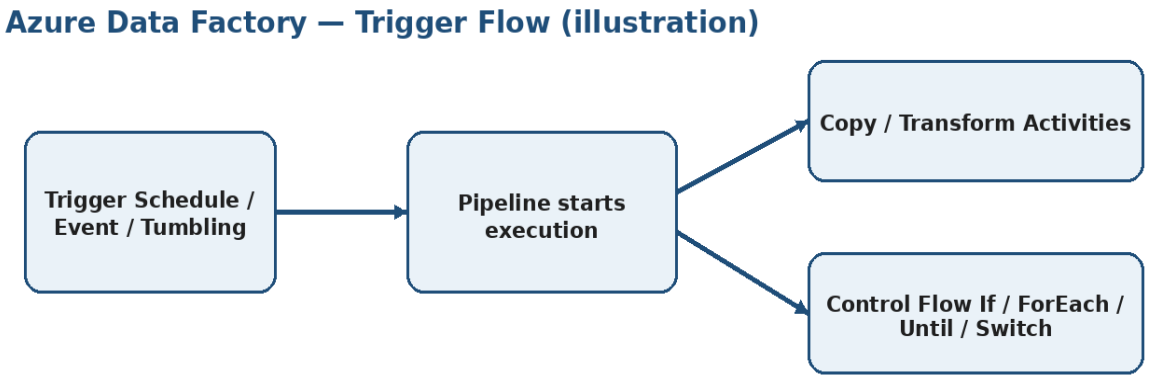
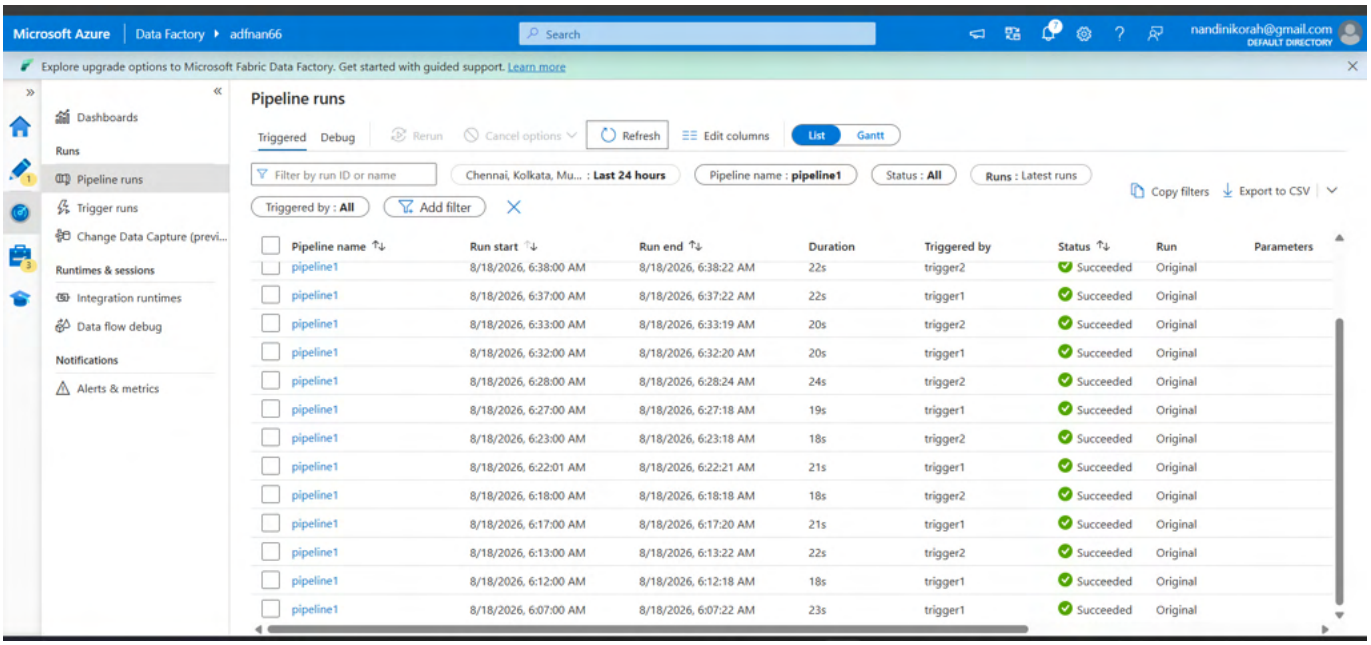
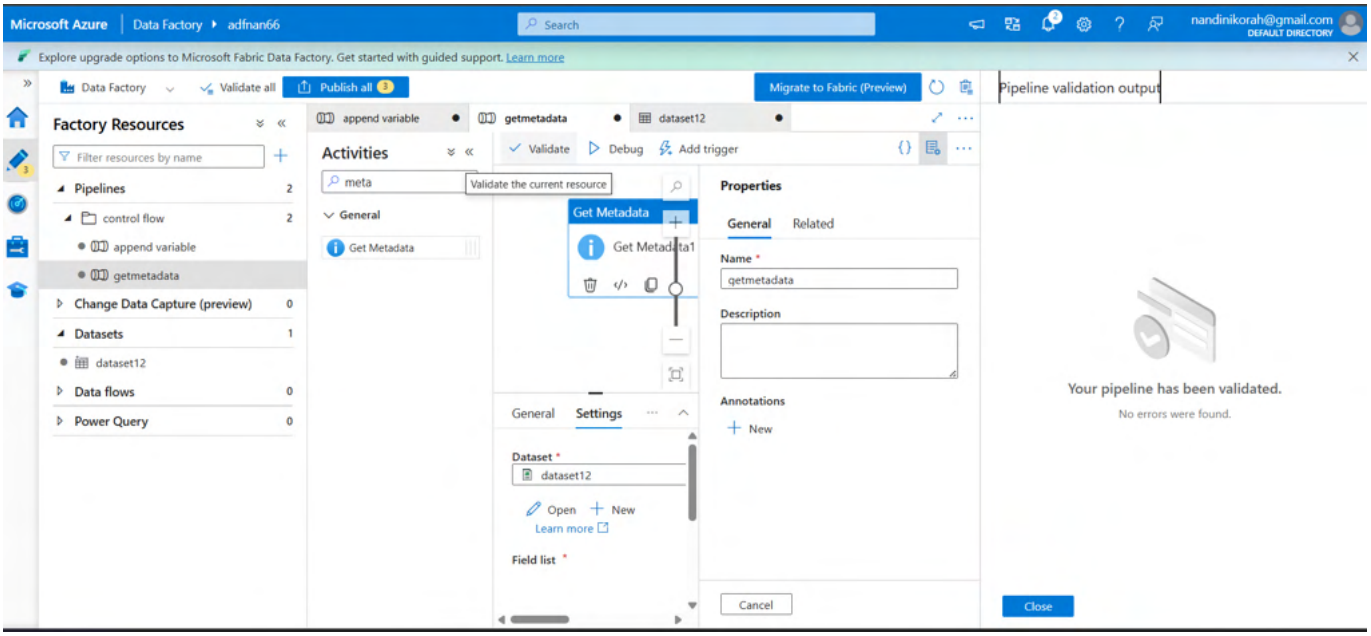


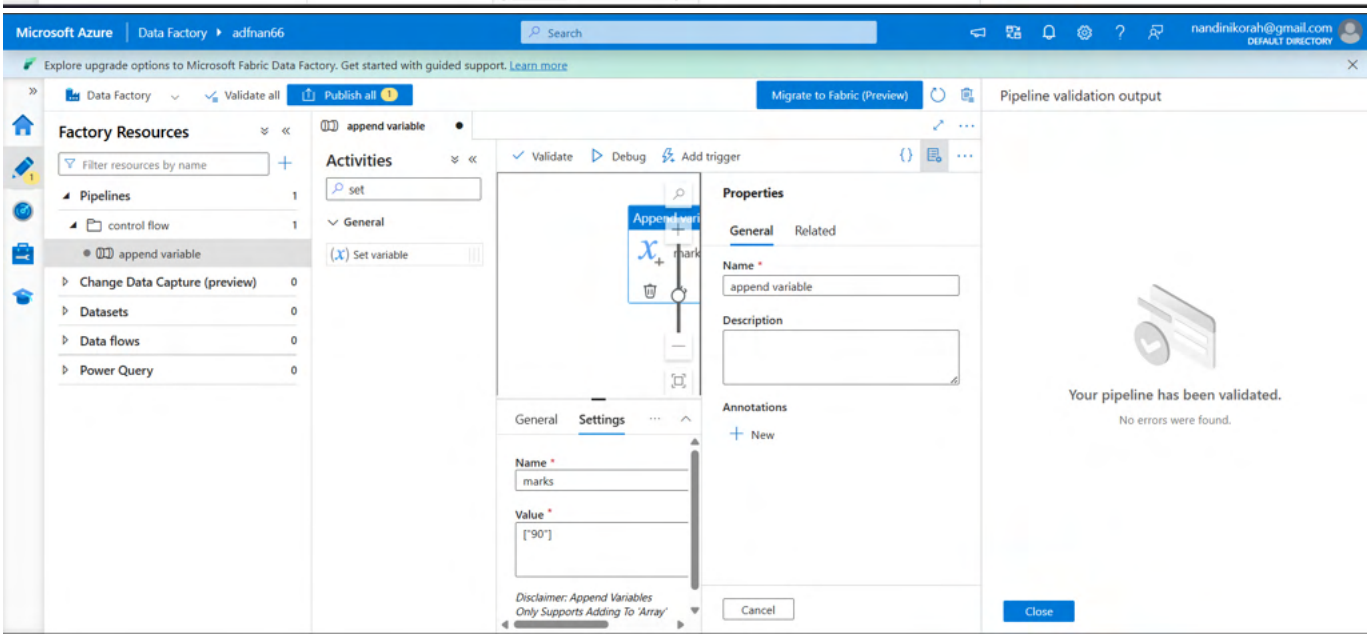
Figure 5. Simplified trigger-to-processing pattern.

16. Validation, Debugging and Publishing

46. Select Validate or Validate all to identify configuration errors.
47. Use Debug to test the pipeline before relying on a live trigger.
48. Review activity output and error messages when a run fails.
49. Publish all changes after testing.
50. Start/activate the trigger.
51. Use Monitor → Pipeline runs to inspect pipeline executions.
52. Use Monitor → Trigger runs to inspect trigger executions.
53. For failed activities, inspect the activity output and dependency path before changing the pipeline.







17. Common Beginner Mistakes

- Creating a trigger but forgetting to publish the changes.
- Creating a trigger but leaving it stopped/inactive.
- Using the wrong expression syntax or missing the @ symbol.
- Referencing an activity output with the wrong activity name.
- Forgetting that trigger date/time values may be represented in UTC/system-variable formats.
- Using ForEach without supplying an array in the Items field.
- Creating an Until loop without a sensible timeout/exit condition.
- Using If Condition when Switch would make several fixed-value cases easier to maintain.
- Testing only the happy path and not testing failure/false/default branches.

18. Microsoft Learn References

- Pipelines and activities — https://learn.microsoft.com/en-us/azure/data-factory/concepts-pipelines-activities
- Create schedule triggers — https://learn.microsoft.com/en-us/azure/data-factory/how-to-create-schedule-trigger
- Create tumbling window triggers — https://learn.microsoft.com/en-us/azure/data-factory/how-to-create-tumbling-window-trigger
- System variables — https://learn.microsoft.com/en-us/azure/data-factory/control-flow-system-variables
- Get Metadata activity — https://learn.microsoft.com/en-us/azure/data-factory/control-flow-get-metadata-activity
- Lookup activity — https://learn.microsoft.com/en-us/azure/data-factory/control-flow-lookup-activity
- ForEach activity — https://learn.microsoft.com/en-us/azure/data-factory/control-flow-for-each-activity
- Wait activity — https://learn.microsoft.com/en-us/azure/data-factory/control-flow-wait-activity

End of guide